IS 107 – Business Intelligence

Laboratory Work 5

Applying DAX Fundamentals in Power BI

Laurence Daleon

Instructor: Joseph A. Vistal | Course: IS 107 – Business Intelligence

GUIDE QUESTIONS

1. Defining DAX For me, the simplest way to explain it is: Excel formulas work on individual cells — they calculate one value at a time and don't understand relationships between tables. DAX, on the other hand, is designed for entire columns and related tables. My thought is that DAX is like Excel formulas but upgraded — it understands your data model, works across multiple tables, and recalculates automatically based on what filter or visual is active on your report.

2. The "Why" of DAX Raw numbers are just static data — they can't answer questions on their own. For me, the gap DAX fills is the ability to create logic on top of that data. For example, the Orders table has Revenue and Cost columns, but nothing tells Power BI to subtract them, group them by customer, or filter them by product. DAX fills that gap by letting us write formulas that define exactly how the data should be calculated, combined, and responded to when a user interacts with the report.

3. The Storage Difference Calculated Columns increase the file size. This is because a calculated column computes and stores a value for every single row in the table at refresh time — in our case, that's 700 rows in Orders. A Measure, by contrast, stores nothing — it only calculates when a visual needs it. For large datasets with millions of rows, using calculated columns for everything would make the file unnecessarily large and slow, which is why it's important to use Measures whenever possible.

4. Context Clues — Filter Context Filter Context is what's currently "active" in the report — which customer is selected, which slicer is clicked, which row is being shown. For me, the clearest example in my dashboard is the Count of Orders measure. When no filter is active, it returns 700 (all orders). But the moment I click on ACME Bites, the filter context narrows and the measure automatically returns 206 — only ACME's orders. The measure didn't change, the context did. That's the power of filter context.

5. The RELATED Function RELATED is necessary when you need a value from a different table while working inside a calculated column. For example, when creating the Has Chocolate column inside the Orders table, I needed to check the product category from the Cookie Types table. RELATED lets you "reach into" the related table to pull that value. For this to work, a relationship must be established in the Model View between the two tables — in our case, Cookie Types and Orders are connected through the Product column.

6. Deconstructing the Formula For Total Sales = SUM(Sales[Sales Amount]):
Measure Name: Total Sales
Function: SUM( )
Table: Sales
Column: Sales Amount
Including the table name is best practice because Power BI can have multiple tables with columns of the same name. Writing Sales[Sales Amount] makes it clear exactly which column you mean, preventing errors and making your formula easier to read and maintain.

7. Data Interpretation A negative Total Profit does not mean the formula is broken — my thought is that it means the formula is working correctly and the business data is telling you that costs exceeded revenue for that category. To verify, I would go to the raw Orders table and check the Revenue and Cost values for that specific category. If the numbers there genuinely produce a negative result, it's a business problem, not a DAX problem. The formula is just reporting what the data says.

8. Optimization A single Measure is far more efficient. Creating a Calculated Column for Profit Margin % would compute and store a value for every row in the dataset — 700 rows in our case — every time the data refreshes, increasing file size. A Measure like

Profit Margin % = [Total Profit] / SUM(Orders[Revenue])

computes nothing until a visual requests it, and then it only calculates the aggregated result for the current filter context. It's lighter, faster, and dynamically correct no matter how the report is filtered.

Visualization

Visual Outputs & Screenshots

Guide Questions| Dashboard Visualizations | Model Relationship Diagram | Measurements

Laurence Daleon

Instructor: Joseph A. Vistal | Course: IS 107 – Business Intelligence

Dashboard Visuals

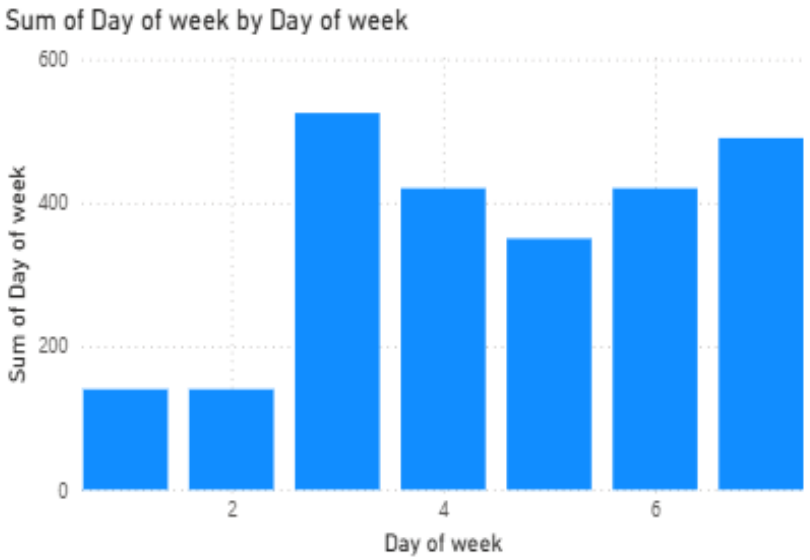
Name	Total Number of Units sold
ACME Bites	329,677
Wholesome Foods	266,890
ABC Groceries	221,023
Park & Shop Convenience Stores	179,448
Tres Delicious	128,769
Total	**1,125,806**

Name	Count of orders
ABC Groceries	132
ACME Bites	206
Park & Shop Convenience Stores	114
Tres Delicious	92
Wholesome Foods	156
Total	**700**

Name	Distinct Customers	Total Profit	Revenue minus Cost
Wholesome Foods	1	639,677.00	639,677.00
Tres Delicious	1	302,199.00	302,199.00
Park & Shop Convenience Stores	1	423,498.50	423,498.50
ACME Bites	1	828,379.00	828,379.00
ABC Groceries	1	523,317.00	523,317.00
Total	**5**	**2,717,070.50**	**2,717,070.50**

Cookie Type	Profit Margin %
Chocolate Chip	60.00%
Fortune Cookie	50.00%
Oatmeal Raisin	56.00%
Snickerdoodle	62.50%
Sugar	58.33%
White Chocolate Macadamia Nut	54.17%
Total	**57.93%**

Dashboard Visuals



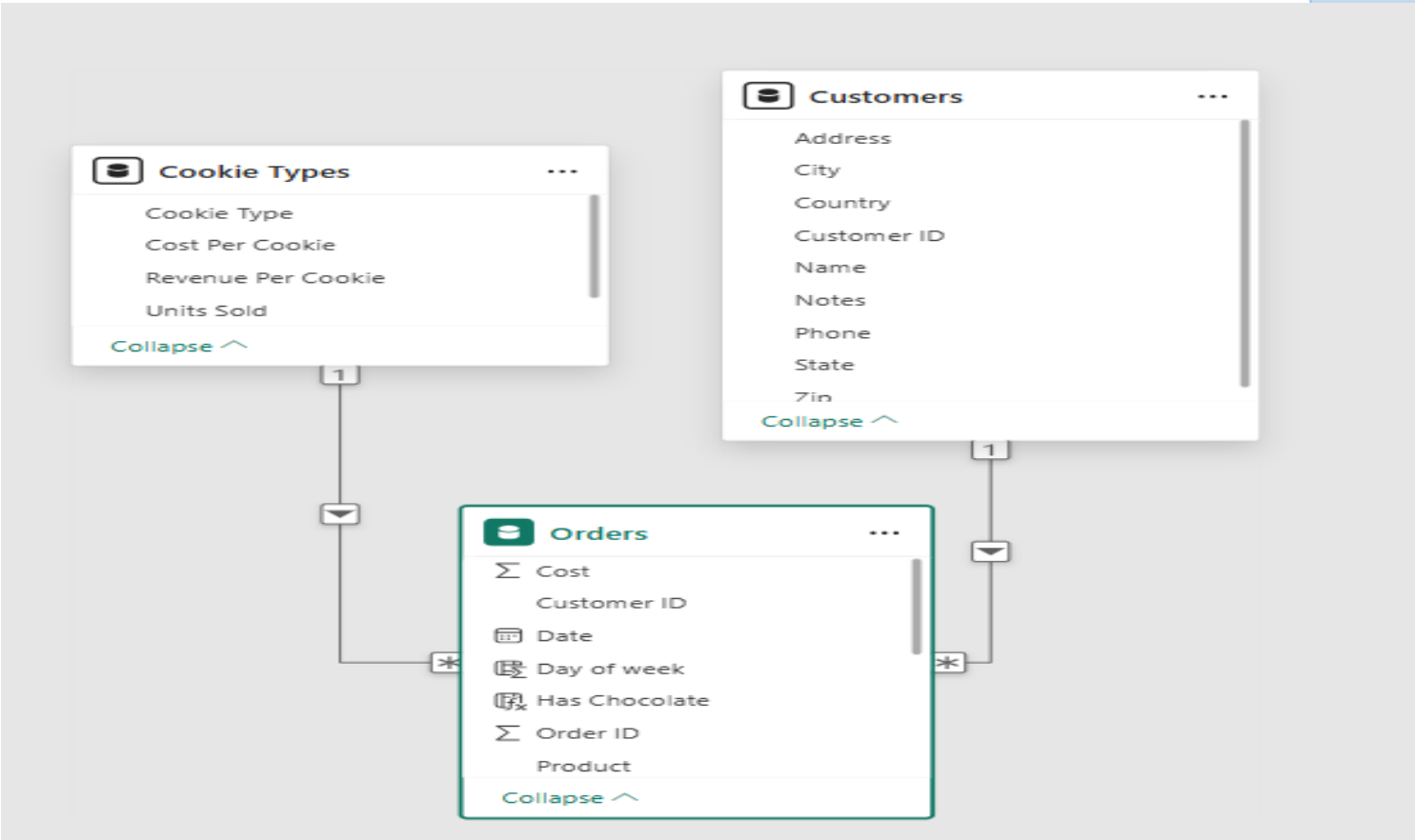
Total Profit 2

2,717,064.38

Chocolate chip with over 500 units

185

Table Relationship



MEASURE FORMULAS

```
Total Profit 2 = sumx('Cookie Types','Cookie Types'[Units Sold]*('Cookie Types'
    [Revenue Per Cookie] - 'Cookie Types'[Cost Per Cookie]))
```

```
Day of week = WEEKDAY(Orders[Date],1)
```

```
Chocolate chip with over 500 units = CALCULATE(COUNTROWS(Orders),Orders[Units Sold]
    >500,Orders[Product]=="Chocolate Chip")
```

```
Profit Margin % = [Total Profit] / sum(Orders[Revenue])
```

```
Count of orders = COUNTROWS(Orders)
```

```
Revenue minus Cost =
SUM('Orders'[Revenue]) - SUM('Orders'[Cost])
```

```
Distinct Customers = DISTINCTCOUNT(Orders[Customer ID])
//This tels me how many customers we have
```

```
Total Number of Units sold = SUM(Orders[Units Sold])
```